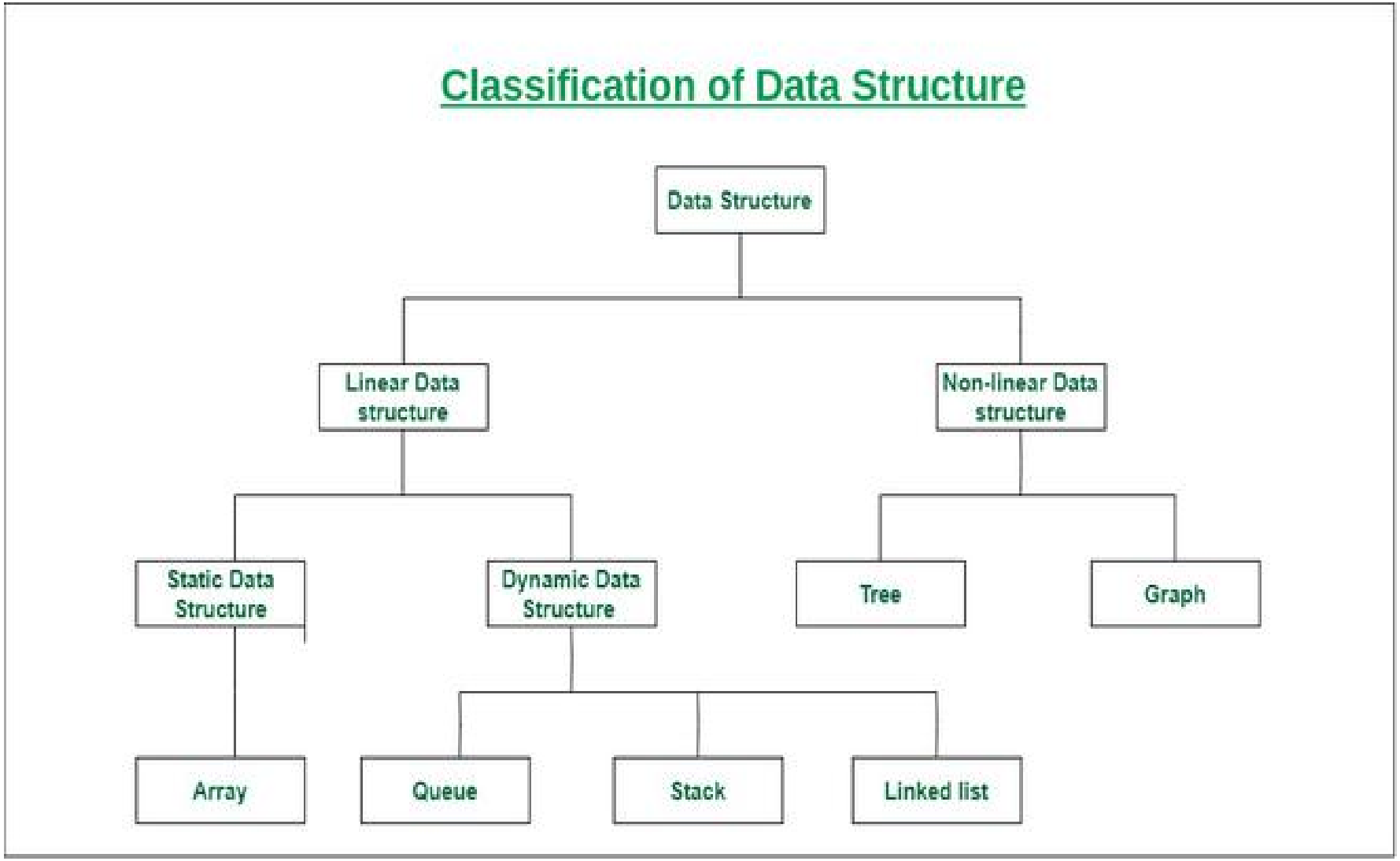
IT12 : Data Structure and Algorithms

What is Data Structure

- A data structure is a storage that is used to store and organize data. It is a way of arranging data on a computer so that it can be accessed and updated efficiently.
- A data structure is not only used for organizing the data. It is also used for processing, retrieving, and storing data.

What is Data Structure



What is Linked List

- A **linked list** is a linear data structure where elements (called **nodes**) are stored in a sequence, but not necessarily in contiguous memory locations.

Each node has two parts:

- **Data** – the actual value stored.
- **Pointer (or reference)** – a link to the next node in the sequence.
- Unlike arrays, linked lists don't need continuous memory blocks, so they allow **dynamic memory allocation**, **efficient insertion and deletion**, but **slower random access**.

- A **linked list** is like a chain of connected items where you can easily insert or remove elements without shifting the rest.

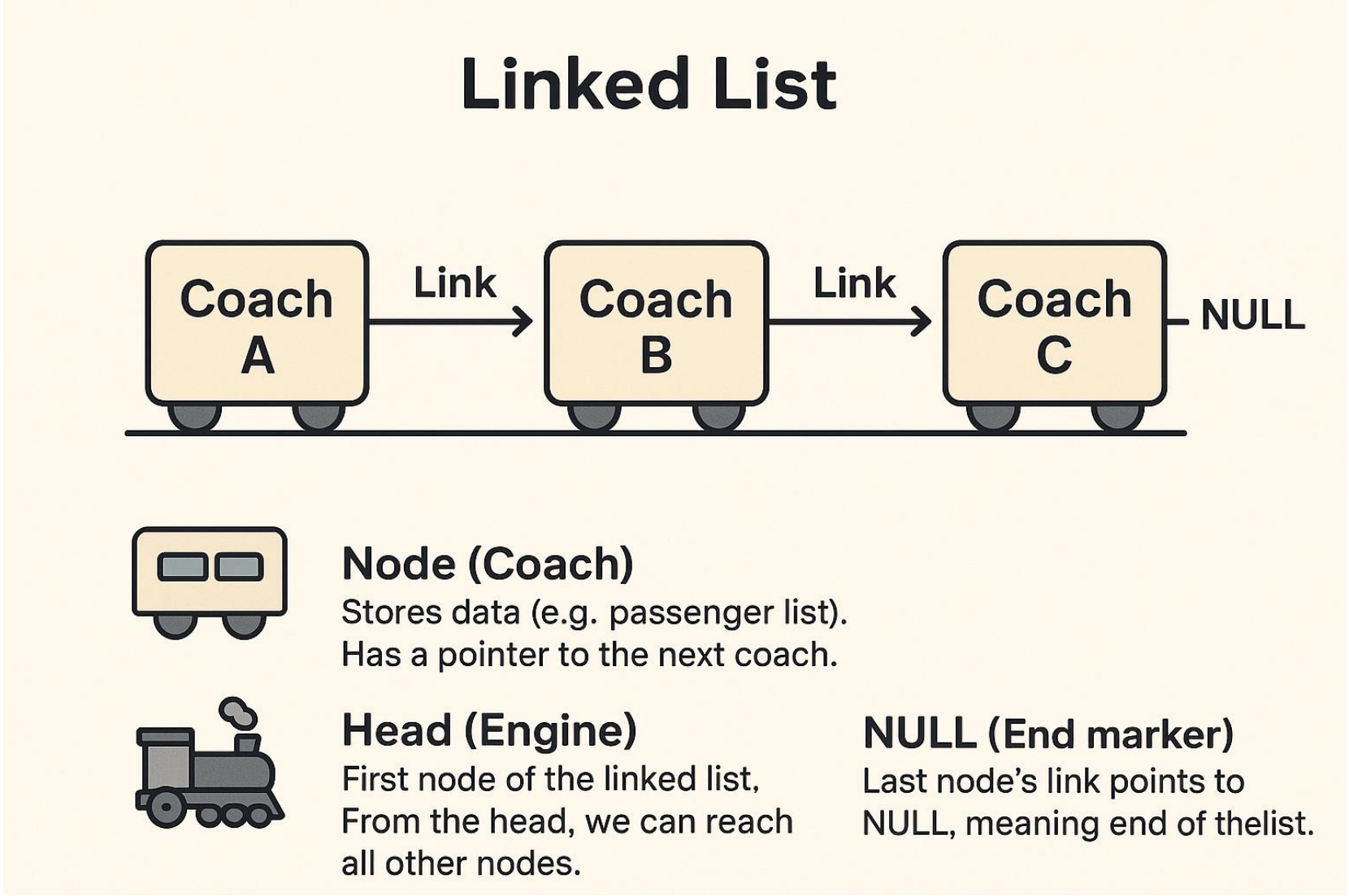
Real Life Examples of Linked List

Example 1: Train Coaches

- Imagine a train: each coach (node) has passengers (data) and a connector (pointer) that links it to the next coach.
- If you want to add a new coach, you just connect it to the existing chain without rearranging all coaches.

- **Example 2: Playlist of Songs**
- Each song (node) contains the track (data) and a link to the next song (pointer).
- You can skip, add, or remove songs without shifting the entire playlist, just by changing links.

Diagram of Linked List



Types of Linked List

Type	Structure	Use Case Example
Singly Linked List	Each node points to **next** node only.	Playlist of songs
Doubly Linked List	Each node has two pointers: **next** and **previous**.	Browser back/forward navigation
Circular Linked List	Last node points back to the **first node**.	Multiplayer games turn rotation

Operations on Linked List

Operation	Description
Traversal	Go through each node from head to end
Insertion	Add a new node at beginning, middle, or end
Deletion	Remove a node
Search	Find an element in the list

Algorithm of Linked List

Steps:

1. **Start**
2. Initialize head = NULL.
3. For each new node you want to add:
    1. Allocate memory for the new node.
    2. Store data in the new node.
    3. Set new_node.next = NULL.
4. If head == NULL (the list is empty):
    1. Make head = new_node (first node).
5. Else (if the list already has nodes):
    1. Traverse from head to the last node (where next == NULL).
    2. Set last.next = new_node.
6. Repeat step 3–5 until all nodes are created.
7. **Stop**

Add node at Beginning Linked List

1.Create a new node

Allocate memory for the new node.

Set new_node.data = value.

2.Link the new node to the current list

Set new_node.next = head (current first node).

3.Update the head

Set head = new_node.

4.Done

The new node is now the first node of the list.

Add node after position Linked List

Steps:

- **Create a new node**
  - Allocate memory for the new node.
  - Set new_node.data = value.
- **Traverse to the given position #** (while count < position and current != NULL)
  - Start from head.
  - Move forward position times to reach the node after which we insert.
  - Let this node be current.
- **Adjust the links**
  - Set new_node.next = current.next
  - Set current.next = new_node
- **Done**
  - Node inserted after the given position

Create Linked List


```
# Node class
class Node:
    def __init__(self, data):
        self.data = data  # store data
        self.next = None  # store reference
to next node

# Linked List class
class LinkedList:
    def __init__(self):
        self.head = None  # initially empty
list

    # Insert at the end
    def append(self, data):
        new_node = Node(data)
        if self.head is None:    # if list is
empty
            self.head = new_node
            return
        last = self.head
        while last.next:  # traverse to the
end
            last = last.next
        last.next = new_node
```

```
# Print the list
    def print_list(self):
        current = self.head
        while current:
            print(current.data, end=" -> ")
            current = current.next
        print("None")

# Usage
ll = LinkedList()
ll.append(10)
ll.append(20)
ll.append(30)
ll. print_list()
```

Add node at Begining


```
# Insert at the beginning
    def prepend(self, data):
        new_node = Node(data)
        new_node.next = self.head
        self.head = new_node
```

```
# Usage
ll = LinkedList()
ll.prepend(100)
```

Add node after given position


```
# Function to insert after a given position
    def insert_after(self, prev_data, new_data):
        temp = self.head
        while temp:
            if temp.data == prev_data:
                break
            temp = temp.next
        if temp is None:
            print(f"Node with data {prev_data} not found.")
            return
        new_node = Node(new_data)
        new_node.next = temp.next
        temp.next = new_node
```

```
# Usage
ll = LinkedList()
ll.insert_after(20, 25)  # insert 25 after node with data 20
```

Delete node


```
def delete_node(self, key):
    temp = self.head

    # Case 1: Head node itself is to be deleted
    if temp and temp.data == key:
        self.head = temp.next
        temp = None
        return

    # Case 2: Search for the key
    prev = None
    while temp and temp.data != key:
        prev = temp
        temp = temp.next

    # If key not found
    if temp is None:
        return

    # Unlink the node
    prev.next = temp.next
    temp = None
```

```
# Usage
ll = LinkedList()
ll.delete_node(20)   # Delete node with value 20

print("After deletion:")
ll.print_list()
```

What is Doubly Linked List

A Doubly Linked List (DLL) is a type of linked list where each node contains:

- Data
- Pointer to Next Node
- Pointer to Previous Node.

This allows **bidirectional traversal**:

- Forward from head to tail
- Backward from tail to head

Real Life examples

- Browser's Back/Forward history
- Undo/Redo feature in text editors
- Navigation menus (move next/previous)

Key Points

- DLL allows movement both ways.
- Slightly more memory needed (extra pointer).
- Insertion & deletion are faster than singly linked lists when node is known.

Advantages of DLL

- Can be traversed in both directions
- Easier deletion (no need to traverse from head to find previous node)
- More flexible compared to singly linked list

Disadvantages :

- Requires extra memory (extra pointer prev)
- Slightly complex implementation

```
class Node:
    def __init__(self, data):
        self.data = data
        self.prev = None
        self.next = None
```

DLL Insertion at the End


```
class Node:
    def __init__(self, data):
        self.data = data
        self.prev = None
        self.next = None

class DoublyLinkedList:
    def __init__(self):
        self.head = None

    def insert_at_end(self, data):
        new_node = Node(data)

        # If list is empty
        if self.head is None:
            self.head = new_node
            return

        # Otherwise, traverse to the end
        temp = self.head
        while temp.next:
            temp = temp.next

        temp.next = new_node
        new_node.prev = temp

    def display_forward(self):
        temp = self.head
        while temp:
            print(temp.data, end=" <-> ")
            temp = temp.next
        print("NULL")
```

What is Circular Linked List

- A **Circular Linked List** is a variation of a linked list in which:
- The **last node points back to the first node**.
- It can be **singly circular** (only next pointer loops back) or **doubly circular** (both next and prev form a circle)

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
```

Advantages of Circular Linked List

- Efficient use of memory (no NULL pointers)
- Can **continuously traverse** the list from any node
- Useful in **real-time applications** like scheduling

Disadvantages of Circular Linked List:

- More Complex Implementation
- Risk of Infinite Loop
- Harder to Debug
- Extra Care for Deletion

CLL Insertion at the End


```
class CircularLinkedList:
    def __init__(self):
        self.head = None

    def insert_at_end(self, data):
        new_node = Node(data)
        if not self.head:
            self.head = new_node
            new_node.next = self.head
          # point to itself
            return

temp = self.head
        while temp.next != self.head:   # reach
last node
            temp = temp.next

        temp.next = new_node
        new_node.next = self.head
```

```
def display(self):
        if not self.head:
            print("List is empty")
            return

        temp = self.head
        while True:
            print(temp.data, end=" -> ")
            temp = temp.next
            if temp == self.head:
                break
        print("(back to head)")
```

What is Stack

- A **Stack** is a linear data structure that follows the **LIFO (Last In, First Out)** principle:
- The **last element inserted** is the **first element removed**.
- Imagine a stack of plates 🍽️ → you put one on top and remove from the top.

Basic Operations

- **Push(x)** → insert an element at the top.
- **Pop()** → remove the top element.
- **Peek/Top()** → view the top element without removing it.
- **isEmpty()** → check if stack is empty.
- **isFull()** (in array-based stack) → check if no more elements can be added.

Real life Example

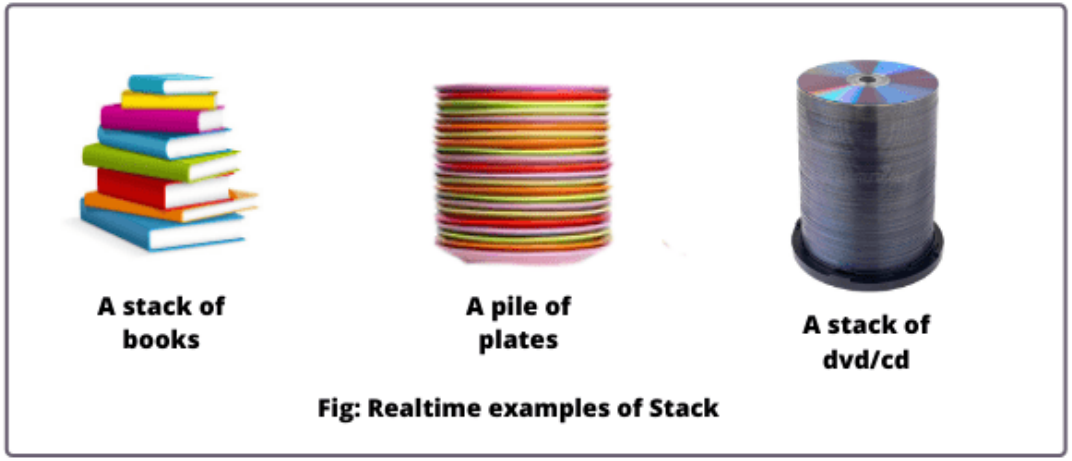
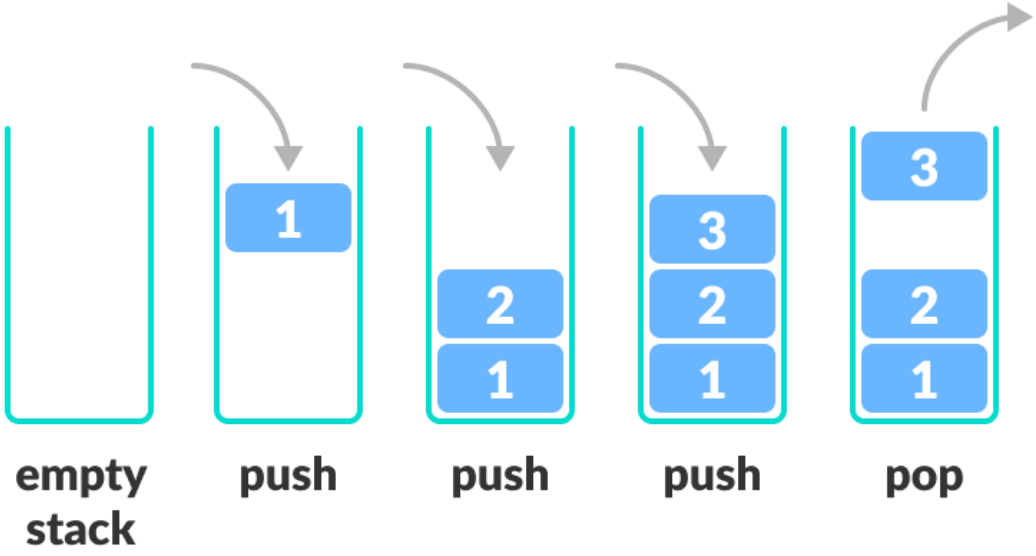


Fig: Realtime examples of Stack



Applications of Stack

- **Expression evaluation** (infix → postfix, prefix)
- **Undo/Redo operations** in text editors
- **Backtracking algorithms** (e.g., maze solving, DFS)
- **Function calls** (call stack in programming languages)
- **Browser history** (Back button)

Advantages & Disadvantages

- **Advantages**
- Simple to implement
- Useful in recursive and LIFO-based problems
- **Disadvantages**
- Limited access (only top can be accessed)
- In array implementation → fixed size (overflow possible)

What is Queue

- A **Queue** is a linear data structure that follows the **FIFO (First In, First Out)** principle:
- The **first element inserted** is the **first element removed**.
- Imagine a queue at a ticket counter → first come, first served.

Basic Operations

- **Enqueue(x)** → insert an element at the **rear**.
- **Dequeue()** → remove an element from the **front**.
- **Front/Peek()** → view the front element without removing.
- **isEmpty()** → check if queue is empty.
- **isFull()** (for array-based queue) → check if no more elements can be added

Types of Queues

- **Simple Queue** – linear queue with single front & rear.
- **Circular Queue** – last element connects to the first for efficient space usage.
- **Priority** Queue – each element has a priority, higher priority dequeued first.
- **Deque (Double-ended Queue)** – insertion & deletion allowed at both ends.

Applications of Queues

- CPU Scheduling
- Print Spooling
- Customer Service Systems
- I/O Buffers
- Breadth-First Search (BFS) in Graphs
- Tasks scheduling
- Admission process
- Payment Process

Advantages of Queue

Maintains Order (FIFO)

•Elements are processed in the exact order they were added.

•**Useful for Scheduling**

•Ideal for CPU scheduling, printer queues, and task management systems.

•**Simple to Implement**

•Can be implemented using **arrays** or **linked lists**.

•**Dynamic in Linked List Implementation**

•No fixed size, unlike array-based queues, so memory is used efficiently.

•**Supports Multiple Variants**

•Circular queue, priority queue, and deque allow flexibility for different applications.

Disadvantages of Queue

Limited Access

Only the **front and rear** elements can be accessed directly.

Wasted Space in Linear Array Queue

After multiple dequeues, empty slots at the front are unused unless circular queue is implemented.

Slightly More Complex in Circular/Deque Variants

Circular queues and double-ended queues require extra logic for pointer management.

Sequential Access

To access a middle element, you may have to traverse from front, which is inefficient

Sample Questions

Doubly linked list node contains _____ pointers
In singly linked list one field is data/info field and the second field is?
A variant of linked list in which the last node of the list points to the first node of the list is?
In doubly linked lists, traversal can be performed?
What kind of linked list is best to answer question like- What is the item at position n?
In singly circular linked list insertion of node at in between position requires modification of?
Linked lists are not suitable to for the implementation of?